Trees and Ensembles

Lesson 7 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

20 November 2026 · reading time about 85 minutes

Contents

Lesson plan . . . 1
1. Why this lesson exists . . . 2
    1.1 The dataset, and the shape a line cannot cover . . . 2
2. Decision trees: splitting on Gini impurity . . . 5
    2.1 The whole algorithm . . . 5
    2.2 Impurity, and the gain a split buys . . . 7
3. Depth is the bias-variance dial . . . 10
4. Reading a tree, and why it doesn't stay still . . . 13
5. Bagging: averaging away the instability . . . 14
    5.1 How much data one bootstrap sample actually sees . . . 14
    5.2 Why averaging reduces variance, and what it cannot fix . . . 16
6. Random forests: decorrelating the trees . . . 17
7. Feature importance, and its limits . . . 18
    7.1 The check: permutation importance . . . 20
8. Gradient boosting: correcting mistakes in sequence . . . 21
9. Boosting for classification: the same idea, a different gradient . . . 23
10. Overfitting in boosting: two knobs that trade off . . . 24
11. Choosing among trees, forests and boosting . . . 26
Summary . . . 27
    Homework . . . 28
    Notation used in this lesson . . . 28
Further reading . . . 28

Lesson plan

Time	Minutes	Segment	Material
0:00–0:10	10	Exercise 6 discussed; the choice this lesson offers	Slides 2–5
0:10–0:32	22	Decision trees: splitting on Gini impurity	Slides 6–20

Time	Minutes	Segment	Material
0:32–0:52	20	Depth is the bias-variance dial	Slides 21–27
0:52–1:12	20	**Notebook 01** — decision trees from scratch	Slide 28
1:12–1:24	12	**Break**	Slide 29
1:24–1:44	20	Bagging and random forests	Slides 30–41
1:44–2:02	18	**Notebook 02** — bagging and random forests	Slide 42
2:02–2:24	22	Gradient boosting	Slides 43–52
2:24–2:42	18	**Notebook 03** — gradient boosting	Slide 53
2:42–3:00	18	The full leaderboard; homework	Slides 54–59
	180	**Total**	**59 slides, 3 notebooks**

---

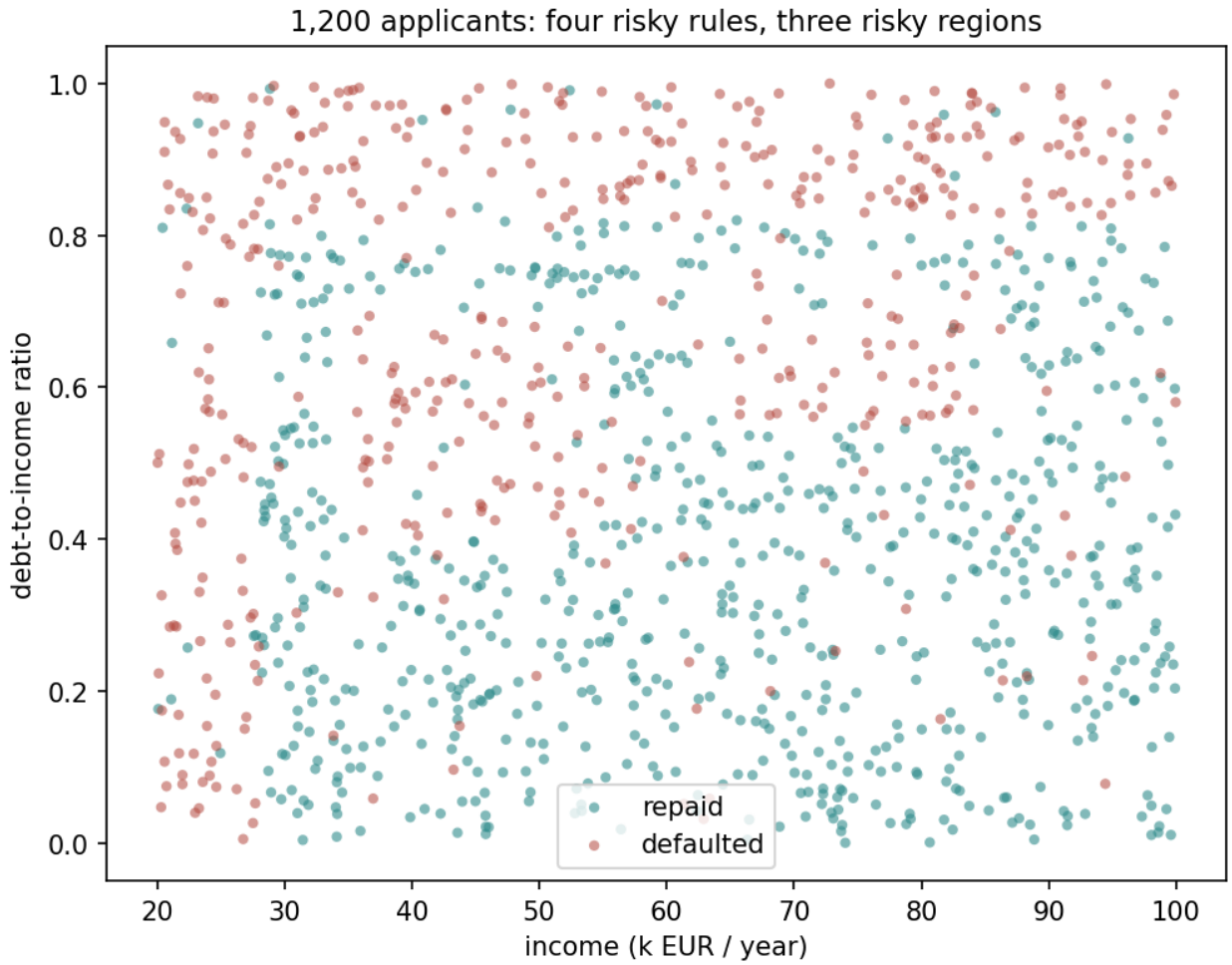
1. Why this lesson exists

Lessons 3 and 4 fit a straight line and made it honest. Lesson 6 offered three ways of bending, remembering or assuming your way around a boundary a line cannot draw. This lesson introduces a fourth family, built on a completely different primitive — a threshold, not a distance or an inner product — and then spends most of its time on the question that actually decides how well trees work in practice: what happens when you build more than one.

A single decision tree is one of the few methods in this course a person without a mathematics background can read end to end. It is also, on its own, a mediocre model: accurate enough to be tempting, unstable enough to be untrustworthy. **Ensembles of trees — bagging, random forests, gradient boosting — are the method most competition leaderboards on tabular data are still won with**, precisely because they turn that instability into their raw material rather than trying to eliminate it.

1.1 The dataset, and the shape a line cannot cover

1,200 loan applicants, two figures each at the point of application: income in thousands of euros a year, and debt-to-income ratio. Whether the loan later defaulted is not a straight function of either. A bank's underwriting logic is closer to a checklist than a formula: an income floor below which almost nobody is approved regardless of anything else, a debt ceiling above which almost nobody is, and two "stressed" combinations in between — income and debt levels that are each unremarkable alone but risky together.



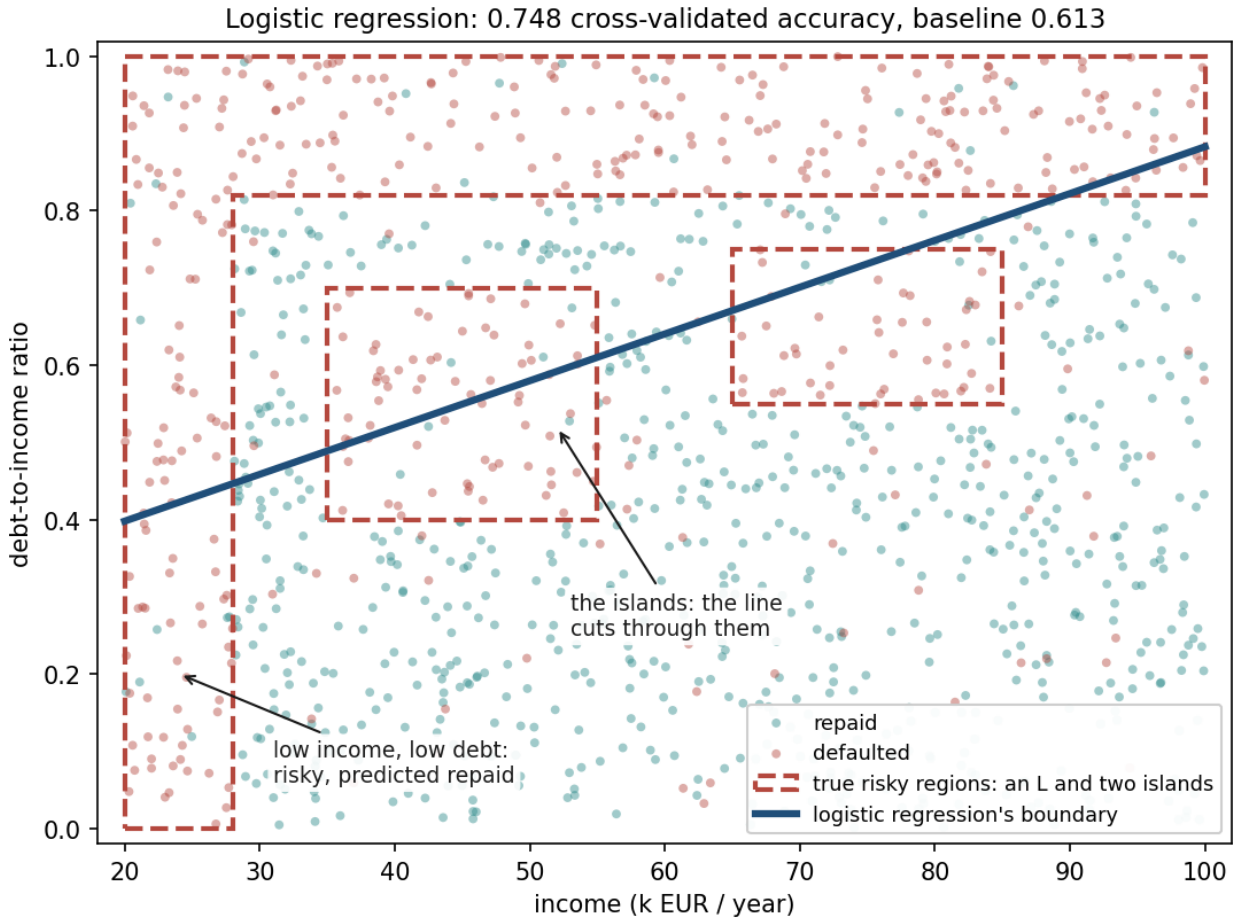
1,200 applicants. Four rules make an applicant risky: low income, or high debt, or one of two rectangles where income and debt are each ordinary but the combination is not. The low-income strip (left edge) and the high-debt strip (top) meet in the top-left corner and form one L-shaped region; the two rectangles stand apart from it. Four rules, three separate regions. No single straight line separates repaid from defaulted, because the class you would have to isolate is not even one connected region.

A predictable mistake is to read the floor and the ceiling as "low income *and* high debt", the top-left corner only. The reading is reasonable, because the two stressed rectangles really are an *and* of two ranges. But the floor and the ceiling each apply on their own, so the risky region they make is the whole L: an applicant with income below the floor is risky at any debt, and one with debt above the ceiling is risky at any income.

7% of labels are flipped after the rule is applied — the noise floor this lesson works against, and the reason no model below should be read as failing when it falls short of 1.000. The ceiling of 0.93 is an average: each label is flipped with probability 7%, and in this particular sample 85 of the 1,200 were, so the true rule itself scores 1,115 / 1,200 = 0.929 on the observed labels. A cross-validated score can land a little either side of 0.93 by chance. 38.7% of applicants defaulted (464 of 1,200), so the majority baseline is 0.613.

Model	Cross-validated accuracy
Always predict the majority class	0.613
Logistic regression	**0.748 ± 0.030**

Unlike lesson 6's pump data — where logistic regression matched the baseline exactly and had learned nothing — here it clears the baseline by a wide margin. Two of the four rules, the income floor and the debt ceiling, are monotonic in one feature — the risk rises steadily as income falls or as debt rises — and a linear boundary captures monotonic effects reasonably well. Where it goes wrong is easiest to see by drawing the line it fits.



The boundary logistic regression draws when fitted on all 1,200 applicants: above the blue line it predicts default, below it repaid. The dashed outlines are the true risky regions. Look at the three places the line disagrees with them: the bottom of the low-income strip, below the line; the right end of the high-debt strip, where the line climbs above the ceiling; and the two islands, which the line cuts straight through.

The line runs from a debt ratio of 0.40 at an income of 20 thousand euros to 0.88 at 100 thousand: almost a threshold on debt alone, tilted slightly. Its three failures are the three places the true rule is not a half-plane. Of the 87 applicants with income below the floor and debt below the ceiling, it predicts 50 as repaid, though the rule makes all of them risky. At high income it misses debt ratios between the ceiling, 0.82, and the line, 0.88. And it cannot wall off the two *interior* islands without

calling part of each healthy and the healthy territory beside them risky. A boundary built from thresholds, applied more than once, can do all three, and that is what the rest of this lesson builds.

---

2. Decision trees: splitting on Gini impurity

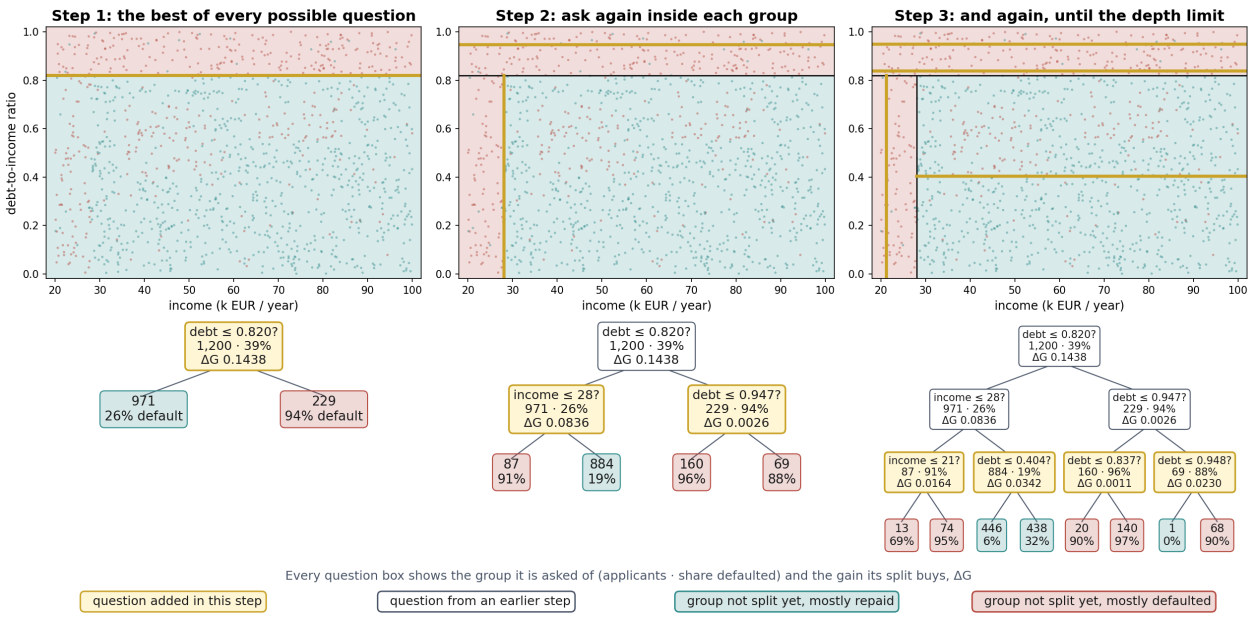
2.1 The whole algorithm

> Find the single (feature, threshold) split that makes the two resulting groups as pure as possible. Apply it. Repeat on each group, independently, until a stopping rule is met.

Nothing is estimated in the sense lessons 3 and 4 used the word — no coefficient, no gradient with respect to a parameter vector. The tree still learns from the data which feature to ask about and where to put each threshold, but it learns them by trying candidates and keeping the best, not by descending a slope. A tree is a sequence of yes/no questions, chosen greedily, and "greedily" matters: at every step the tree takes whichever split helps most *right now*, with no mechanism for looking ahead to a split that would help more two levels down.

A predictable mistake is to read the algorithm as "one feature per level, applied to all the examples, then the next feature". It is a natural reading: a drawn tree looks like a fixed sequence of questions, and the first question really is asked of everyone. It is wrong twice over. Nobody chooses the feature: at every node the tree tries every feature and every threshold and keeps the best pair. And after the first split each group is searched on its own, so the two groups can be asked different questions, and a feature can come back as often as it helps — in the figure below, one path asks about debt three times.

The recursion is easiest to see run. Three rounds of it on the loan data:



Each column is one more round of "find the best question, apply it, repeat inside each group". On top, the cuts in the plane of the data, the ones added in that round in gold; underneath, the tree asking the same questions. Every box shows its applicants and their default rate, a question as well as a group not yet split: when a group is split its box becomes the question asked of it, so the 971 of step 1 reads "income 28?" in step 2, with 971 under it, and splits into 87 + 884. Step 2 asks

a different question in each group, because each is searched on its own. Step 3 shows greed: one cut, at 0.948 — a thousandth above the one at 0.947, so the two gold lines overlap — sets aside a single applicant from 68, another splits 160 into 90% and 97% default and changes no prediction, and neither interior island has been found yet. Each question box also shows the gain its split buys, ΔG (section 2.2): in step 2, 0.0836 for the income question and 0.0026 for the debt one, whose two groups are both still mostly defaulted, so that split changes no prediction either.

How little the later rounds buy is worth counting. The four splits of step 3 change exactly one prediction in 1,200: training accuracy is 0.8400 at depth 2 and 0.8408 at depth 3. And that one applicant is a flipped label: income 23.22, below the floor, and debt 0.9475, above the ceiling, so both rules say default, yet the label says repaid. The tree spent a split memorising one noisy label, which is section 3's story about depth in miniature. It splits anyway because it judges impurity, not decisions: any split whose ΔG is above zero is taken.

The stopping rules act branch by branch. A group that is pure, or too small to split, or at the depth limit, stops its own branch; the other branches carry on, and the tree is finished when every branch has stopped. Without a depth limit every branch runs until its group is pure.

The loop, written out. Everything in this section and the next, in the order the code runs it (notebook 1's `build_tree` and `best_split`):

```
def grow(g):
  if stop(g): return leaf(g)
  gain = {c: ΔG(g, c) for c in splits(g)}
  best = max(gain, key=gain.get)
  if gain[best] <= 0: return leaf(g)
  L, R = split(g, best)
  return node(best, grow(L), grow(R))
```

Line by line: `g` is the group being grown; `stop(g)` is true when the group sits at the depth limit, is pure, or has fewer than 2 examples; `leaf(g)` predicts the group's majority class; `splits(g)` lists every candidate, a feature with one of its midpoint thresholds; `ΔG(g, c)` is the gain of section 2.2 on the two groups candidate `c` would make; `max(gain, key=gain.get)` picks the candidate with the largest gain; `split` makes the two groups, `L` and `R`; `node` keeps the question and its two subtrees.

ΔG, defined in section 2.2, does two jobs, both at one node at a time: it ranks that node's candidates, and it decides whether the node is split at all. Each child then runs the whole loop again on its own examples, with its own impurity in the parent's place and its own size as m; gains are never added up across nodes or compared between them in this loop (section 4's `max_leaf_nodes` is the exception, and compares them on purpose).

What the two constructors return is worth saying, because the code hides it. `leaf(g)` is a new object that keeps one thing, the majority class of `g`, as its prediction; it does not keep the examples. `node(best, grow(L), grow(R))` holds the question and the two subtrees. Python evaluates the arguments first, so both recursive calls run all the way down before the node that holds them is built: the tree is assembled from the leaves upward, depth first. At the root, `node((debt, 0.82), grow(the 971), grow(the 229))`. And `max(gain, key=gain.get)` returns the candidate with the largest gain, not the gain itself. **Two predictable mistakes** come from reading it too fast: "the function returns `g`" (it returns a leaf built from `g`) and "`max` gives the best gain" (it gives the best candidate).

Training builds the tree; inference walks it. To predict for a new applicant, start at the root;

at each question go left if the applicant's value is at or below the threshold, right otherwise, until a leaf, and return its class (notebook 1's `predict_one`). The two costs are lopsided: training scores every candidate at every node, while a prediction is at most `max_depth` comparisons, 8 for the best tree here, however large the training set was. Lesson 6's k-nearest neighbours is the opposite: nothing to train, and every prediction compares the new point with all the training points.

2.2 Impurity, and the gain a split buys

"As pure as possible" needs a number. For classification, the standard choice is the **Gini impurity** of a group of m_g examples drawn from C classes:

$$G = 1 - \sum_{c=1}^{C} p_c^2$$

where p_c is the fraction of the group belonging to class c. The classes are the labels a group can hold: here $C = 2$, defaulted and repaid, and the two shares add up to 1. Slide 8's right-hand group of step 1, before it is split again, holds 229 applicants of whom 215 defaulted, so $p = 215/229 = 0.939$ for defaulted and $14/229 = 0.061$ for repaid, and $G = 1 - (0.939^2 + 0.061^2) = 1 - 0.885 = 0.115$: nearly pure. For two classes this simplifies to $2p(1-p)$: expand $(1-p)^2$ as $1 - 2p + p^2$, and the two p^2 terms and the two 1s cancel, leaving $G = 1 - p^2 - (1 - 2p + p^2) = 2p - 2p^2$. It is 0 when the group is pure ($p \in \{0, 1\}$) and maximal, 0.5, at $p = 0.5$.

What that number is, exactly. G is not a probability of misclassification. It is the probability that two examples drawn at random *with replacement* from the group, and labelled according to the group's own class frequencies, would **disagree** — and the formula is that statement written out. Draw twice: the two draws agree on class c with probability p_c^2, so they agree at all with probability $\sum_c p_c^2$, and they disagree with the rest:

$$P(\text{disagree}) = 1 - \sum_{c=1}^{C} p_c^2 = G$$

A pure group cannot disagree with itself, which is why $G = 0$ there.

A note of history. The name is Italian. Corrado Gini (1884–1965) was a statistician and the first president, in 1926, of the Istituto Centrale di Statistica, the body that became ISTAT. $1 - \sum_c p_c^2$ is his index of heterogeneity ("mutabilità", 1912); Breiman, Friedman, Olshen and Stone made it the splitting criterion of their CART trees in 1984, and scikit-learn kept it. It is not the Gini coefficient of income inequality, his better-known index — the same man, a different quantity.

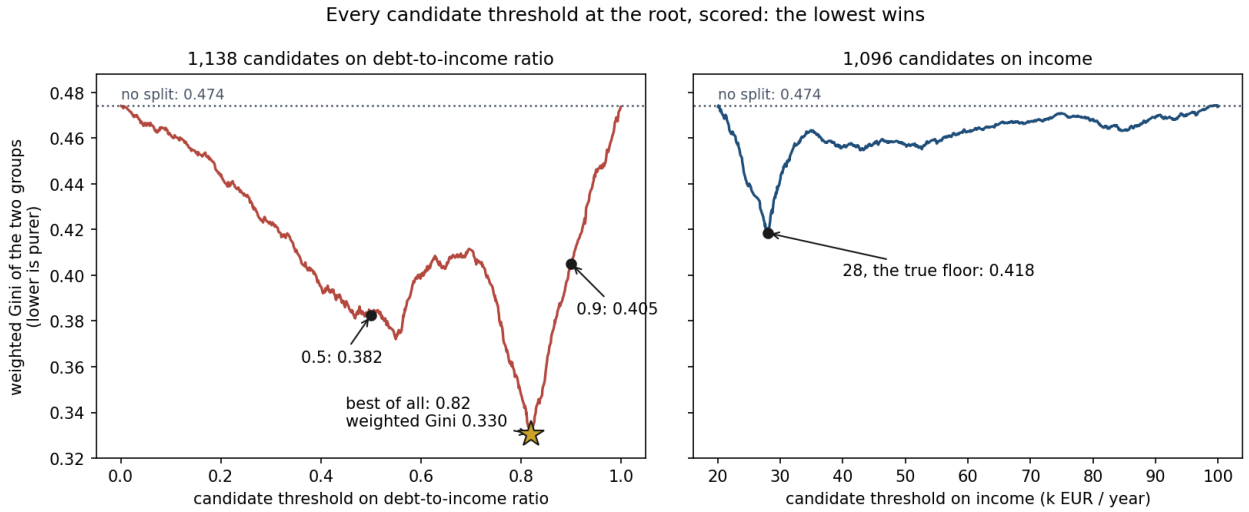
A candidate split divides a parent group of m_P examples into a left child of m_L examples and a right child of $m_R = m_P - m_L$. Its value is the reduction in impurity, weighted by how the group is divided:

$$\Delta G = G(\text{parent}) - \left(\frac{m_L}{m_P} G(\text{left}) + \frac{m_R}{m_P} G(\text{right}) \right)$$

For a continuous feature, the tree scans every feature and, within each, every midpoint between two adjacent sorted values as a candidate threshold, and keeps whichever (feature, threshold) pair

maximises ΔG. It then repeats the entire search independently inside each child. This is what makes the algorithm greedy: ΔG is evaluated one split at a time, never for a pair of splits jointly, so a feature that only helps in combination with a second split further down can be — and in this dataset's two interior islands, is — invisible to a shallow tree.

Infinitely many thresholds, finitely many splits. A threshold is a real number, so there are infinitely many to choose from, but they make only finitely many different splits: any two thresholds between the same two neighbouring values send exactly the same examples left. At the root, two neighbouring debt ratios in the data are 0.8198 and 0.8205; a threshold of 0.8199, 0.8200 or 0.8204 makes the same 971 / 229 split. One threshold per gap is therefore enough, and the convention is the midpoint, $(0.8198 + 0.8205)/2 = 0.82015$, which keeps it as far as possible from both values seen in training. A feature with k distinct values has $k - 1$ gaps: debt ratio has 1,139 distinct values here and income 1,097, so the root search scores $1{,}138 + 1{,}096 = 2{,}234$ candidates. The search is exact, not an approximation — the predictable mistake is to assume it must be approximate because thresholds are continuous.



Every candidate threshold at the root, scored by the weighted Gini impurity of the two groups it makes (lower is purer; the dotted line, 0.474, is no split at all). Look for the lowest point on either curve: debt at 0.82, 0.330. Debt at 0.5 leaves both groups mixed (0.382); income's best threshold, near the true floor of 28, only reaches 0.418.

A predictable mistake is to read "purest" as "the most defaults on one side". The score counts proportions, each weighted by its group's size, not numbers of defaults. Debt at 0.9 makes a right-hand group almost as pure as 0.82 does — 129 applicants at 92% default against 229 at 94% — but leaves the large left group more mixed, 1,071 at 32% against 971 at 26%, and its weighted impurity is 0.405 against 0.330. A group pure in "repaid" counts as much as one pure in "default": $2p(1-p)$ does not ask which class is common.

Worked example. On the full 1,200 applicants, $p = 0.387$ defaulted, so $G(\text{parent}) = 2 \times 0.387 \times 0.613 = 0.474$. The best root split found by exhaustive search is `debt_ratio <= 0.82` — recognisably the debt ceiling the data was built with — and the from-scratch implementation below confirms this is exactly the split scikit-learn's `DecisionTreeClassifier` also finds. More than that: at every depth tested the two make the same prediction for every one of the 1,200 applicants. The check runs on the training data on purpose — it tests the code, whether two programs make the same decisions from the same rows, not how well the tree will do on applicants it has not seen. Identical

predictions do not by themselves prove the two trees are the same node by node, since two different trees can agree on every row they are shown; the notebook therefore also prints the root split from both, and it is the same.

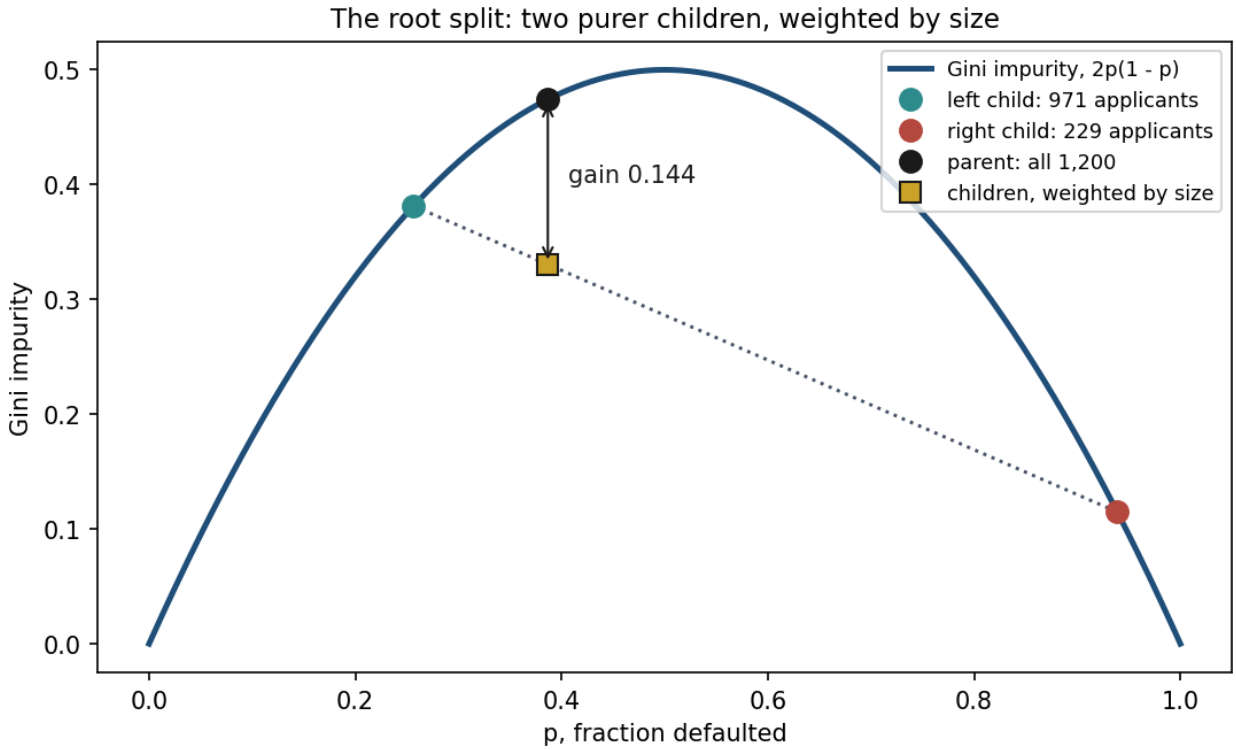
What that split buys, in the formula above. It sends 971 applicants left and 229 right. The rule first: count the defaults in each group, divide by its size to get p, and put p into $2p(1-p)$:

Group	Applicants m	Defaulted	p = defaulted / m	$G = 2p(1-p)$
Parent	1,200	464	464 / 1,200 = 0.387	$2 \times 0.387 \times 0.613 = 0.4743$
Left, `debt_ratio <= 0.82`	971	249	249 / 971 = 0.256	$2 \times 0.256 \times 0.744 = 0.3814$
Right	229	215	215 / 229 = 0.939	$2 \times 0.939 \times 0.061 = 0.1148$

The defaults add up, $249 + 215 = 464$, as they must. (The G column is printed from the unrounded p, which is why its fourth decimal can differ from the product of the rounded values shown.) The two children are weighted by their share of the parent, 971/1,200 = 0.809 and 229/1,200 = 0.191: $0.809 \times 0.3814 + 0.191 \times 0.1148 = 0.3086 + 0.0219 = 0.3305$, below the parent's 0.4743. In one line:

$$\Delta G = 0.4743 - \left(\tfrac{971}{1200} \times 0.3814 + \tfrac{229}{1200} \times 0.1148\right) = 0.4743 - 0.3305 = 0.1438$$

Two predictable mistakes in this arithmetic. The first is to subtract the weighted average from 1 instead of from the parent's impurity: $1 - 0.3305 = 0.6695$, larger than the parent's whole impurity. It is natural, because the Gini formula itself starts with "1−"; the check is that a split cannot remove more impurity than the parent had, so $\Delta G \leq G(\text{parent})$, and with two classes never above 0.5. The second is to rebuild a count from a rounded percentage: the figure in section 2.1 prints the left group as 26% default, and $0.26 \times 971 = 252$, not 249 — the true share is 249/971 = 25.6%. The counts come first.



The Gini curve $2p(1-p)$ with the parent (black) and the two children (teal and rust). The children's size-weighted average (gold) lies on the straight line between them, and the curve bulges above every such line — so any split that separates the classes at all lowers the weighted impurity. The gap, 0.144, is the gain; the tree takes the split whose gap is largest. The square sits exactly under the parent because the children's p, averaged with the same weights, gives the parent's back — with counts it is exact, $\frac{971}{1200} \cdot \frac{249}{971} + \frac{229}{1200} \cdot \frac{215}{229} = \frac{249+215}{1200} = \frac{464}{1200} = 0.387$ — because a split moves defaulters between groups, it does not create any.

> **Try this:** using `Notebooks/loan_data.py`, compute G for the group of applicants with `income_k < 28` (the income-floor rule alone) and compare it with G for the full dataset. It should be substantially lower — that subgroup is disproportionately defaults — which is exactly why the tree is willing to spend a split isolating it.

---

3. Depth is the bias-variance dial

Lesson 6 turned k in k-nearest neighbours and watched bias trade directly against variance. A tree's equivalent dial is **depth**: how many questions any one applicant may be asked on the way from the root to a leaf, where the tree stops and predicts a class by majority vote. Depth counts questions along one path, not in the whole tree: a tree of depth 2 can ask three questions in all, one at the root and one in each of its two branches.

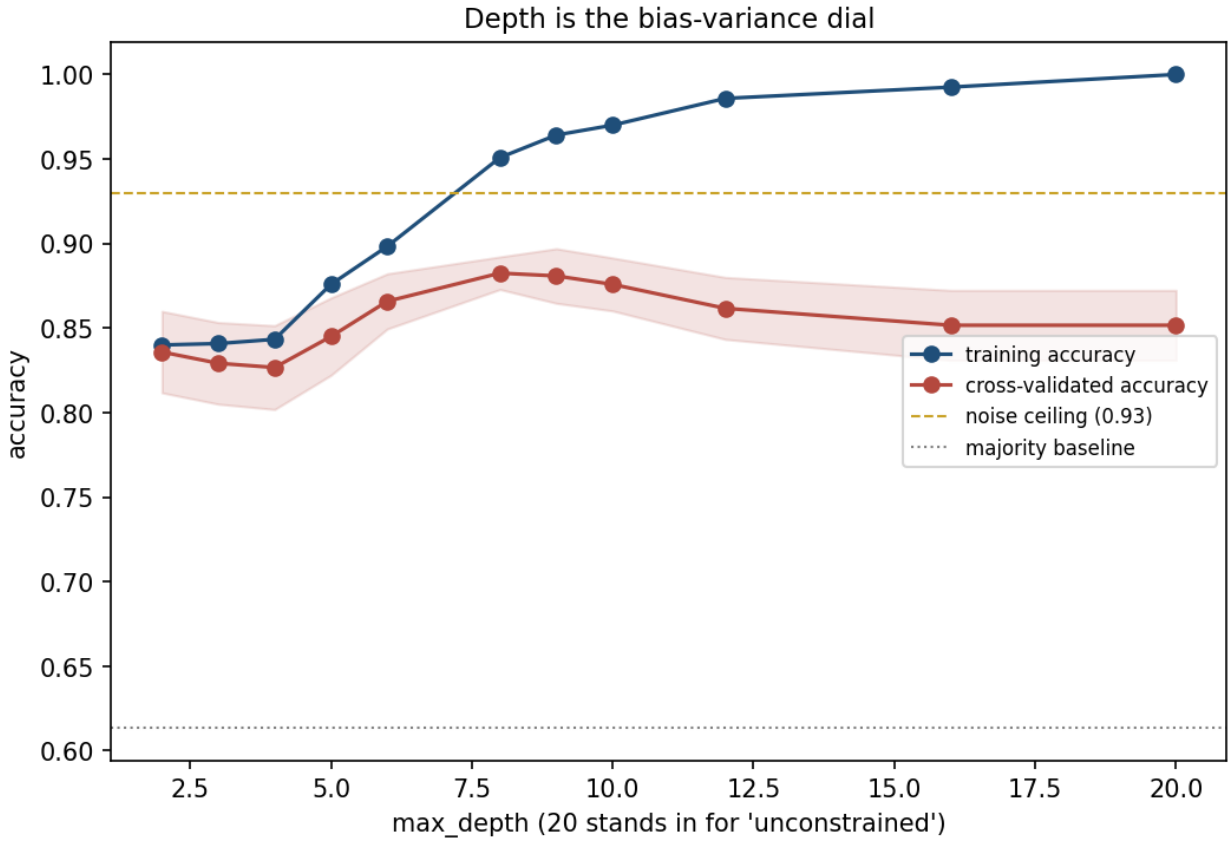
Shallow tree. Few questions, so only the coarsest structure can be described. High bias, low variance.

Deep tree. Enough questions to isolate almost any subset of the training data, down to single points. At full depth, training accuracy is **exactly 1.000**, even on a dataset with no structure at

all: keep splitting and every leaf is eventually pure, at worst one example, which it then "predicts" perfectly. The one exception is two examples with identical feature values and different labels, which no split can separate; with two continuous features drawn at random, as here, that does not occur. That number demonstrates the model class is flexible enough to memorise, not that anything has been learned — the same reading lesson 6 gave $k = 1$'s training accuracy of 1.000.

`max_depth`	Training accuracy	Cross-validated, 5 folds (mean ± standard deviation)
2	0.840	0.836 ± 0.024
4	0.843	0.827 ± 0.025
6	0.898	0.866 ± 0.016
8	0.951	**0.882 ± 0.010**
9	0.964	0.881 ± 0.016
12	0.986	0.862 ± 0.018
16	0.993	0.852 ± 0.021
unconstrained	**1.000**	0.852 ± 0.021

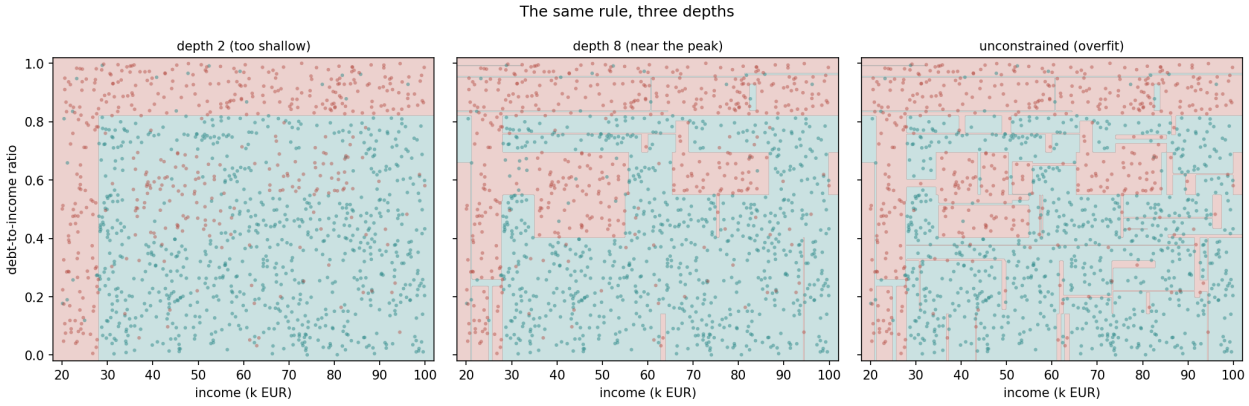
The cross-validated column is lesson 5's five-fold stratified cross-validation: the 1,200 are cut into five folds of 240 with the same default share, and five times the tree trains on 960 and is scored on the 240 it did not see. At depth 8 the five scores are 210, 211, 215, 214 and 209 correct out of 240, so the mean is $(0.875 + 0.879 + 0.896 + 0.892 + 0.871)/5 = 0.882$, and the ± is their standard deviation, 0.010: how much the score moves from one fold to another, not a confidence interval.



Training accuracy climbs monotonically towards 1.000. Cross-validated accuracy rises, peaks at depth 8, and then falls back — even though the model keeps getting more flexible. Past the peak, every extra split is spent fitting some of the 7% of labels this dataset deliberately flips, and a flipped label has no pattern to learn; a tree that classifies it "correctly" on the training set has memorised it, not generalised. The shaded band is ± one standard deviation across the five folds; "20" on the axis stands for the unconstrained tree, whose real depth is 24.

Two ways to see the overfitting. *Who votes in a leaf.* A leaf predicts the majority of its applicants. At depth 2 the tree has 4 leaves with a median of 123.5 applicants each, a vote of a hundred; the unconstrained tree has 156 leaves with a median of 2, and 72 of them hold a single applicant — a prediction made from one person's label, lesson 6's $k = 1$. *The line at 0.929.* The true rule itself scores $1{,}115/1{,}200 = 0.929$ on these labels, because 85 were flipped. A training accuracy above that line means the tree agrees with the labels better than the truth does, so it must be reproducing flipped labels: depth 8 is just past it at 0.951, and the unconstrained tree has learned all 85. The ceiling (the gold line) is the best a model can expect on *new* applicants, whose flipped labels it cannot know; on its own training data it can do better by memorising, which is why the training curve crosses the line and the cross-validated one never does. **A predictable mistake** is to read a training score above the ceiling as a bug: impossible on new data, expected on training data.

Read the two columns against each other, never in isolation. The peak sits at depth 8 — deep enough to isolate both stressed islands, shallow enough to leave the flipped labels as errors rather than as leaves of their own. The unconstrained tree **loses 3 points of cross-validated accuracy relative to the peak** while gaining nothing but a training score nobody should have trusted in the first place.



The same rule at three depths. At depth 2 every applicant answers at most two questions. The tree asks three in all — it is step 2 of the figure in section 2.1 — but the third, on the high-debt side, splits 229 applicants into two groups both predicted "default", so it changes no prediction and draws no line. The boundary shows only the income floor and the debt ceiling — the two biggest, easiest-to-find regions — and neither interior island is visible at all. At depth 8 both islands appear, close to the rectangles of the generating rule though not exact, and a few thin slivers have already started — its training accuracy, 0.951, is past the 0.929 line. At full depth the boundary has grown a fringe of tiny rectangles chasing individual flipped labels, in both colours — rust in healthy territory, teal inside the risky strips: not a smoother version of the depth-8 boundary, a noisier one.

The predictable mistake, and why the instinct is sound. It is entirely reasonable to expect a more flexible model to do at least as well as a less flexible one on held-out data — more options can

only help, surely. The instinct is sound for *bias*: a deeper tree can represent anything a shallower one can, plus more. It is wrong for the quantity that is actually measured, because flexibility increases variance at the same time, and past some point the added variance costs more than the reduced bias saves. The course has shown this curve bend before — lesson 3's test error turned back up as the polynomial degree grew, lesson 6's as k shrank towards 1 — and it will not be the last time.

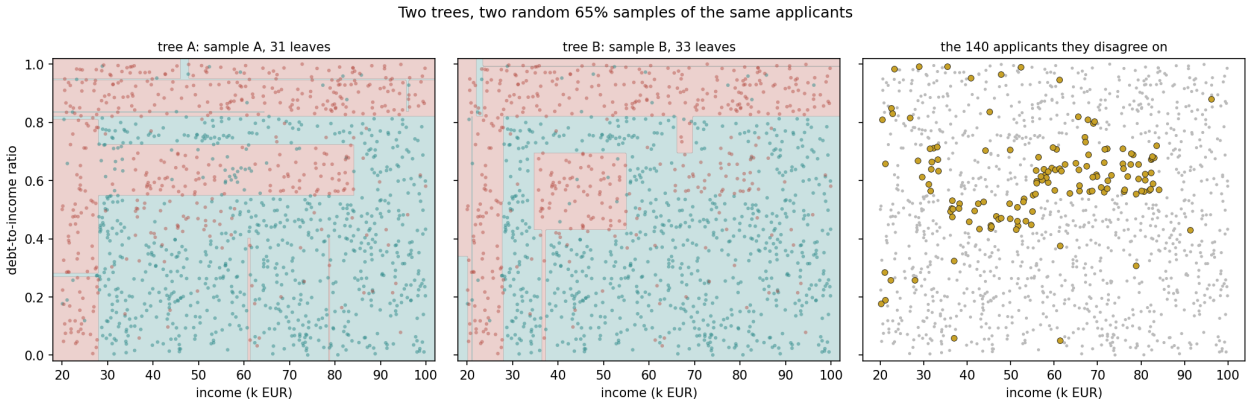
---

4. Reading a tree, and why it doesn't stay still

The two knobs scikit-learn offers limit different things. `max_depth` is a **cap on every path**: each node above the cap is split if it can be, whether or not that particular split is worth having — the depth-2 tree of section 3 spent one of its three questions on a split that changed no prediction. `max_leaf_nodes` is a **budget of leaves** for the whole tree, spent **best-first**: the tree always expands whichever current leaf offers the largest gain — compared across all the current leaves, each leaf's ΔG weighted by its share of all the examples — so a small budget of splits goes to the ones that matter most. It is the one place where gains at different nodes are compared. For a tree meant to be read rather than merely scored, that is the better knob.

With `max_leaf_nodes = 9` the tree reaches 90.8% training accuracy in nine leaves and, more to the point, **0.901 cross-validated** ($\pm$ 0.020) — above the depth-8 tree's 0.882 with its 71 leaves. Nine leaves spent where they gain most beat 71 spent evenly, because this dataset's rule is made of axis-aligned rectangles, the very shape a few well-placed thresholds draw. (The 9 was chosen for readability, not by trying budgets on the folds.) Every threshold is recognisable from the generating rule: 28 for the income floor, 0.82 for the debt ceiling, 34.92 and 58.00 tracing the outline of the first stressed island. That printout is the entire model — a sequence of if/else statements a loan officer could apply by hand — and it is the thing k-nearest neighbours and support vector machines cannot offer at all. For a decision that has to be justified to the person it affects, that can matter as much as accuracy; this lesson's companion reading takes the point further.

The cost is **instability**. An early split near the root reroutes everything beneath it, so a small change in the training data can change the tree's shape. Two trees at `max_depth = 6`, each fit to its own random 780 of the 1,200 applicants — drawn **without replacement**, so no one twice, like cards dealt without putting them back; the two samples share 499 — find the debt ceiling within 0.001 of each other: the evidence for it is overwhelming, so it barely moves. But they grow 31 and 33 leaves and give **opposite verdicts to 140 of the 1,200 applicants** (11.7%): for those 140, the answer depends on which sample the bank happened to train on.



The two trees' boundaries and, right, the 140 applicants they disagree on. Look at where the gold points sit: almost all around the two islands, where each split was decided by a few dozen applicants. The strips, decided by hundreds, agree. The strongest split is stable because the signal behind it is unambiguous; the weaker splits, each supported by far fewer points, are not, and they are most of what determines a tree's final shape.

A single tree's instability looks like a flaw. It is the raw material sections 5–8 turn into a strength.

---

5. Bagging: averaging away the instability

If a tree's mistakes are somewhat random — different across resamples of the same data, as section 4 just measured — then many trees, each grown on its own resample and then averaged, should have mistakes that partly cancel. That is **bagging** (**b**ootstrap **agg**regat**ing**):

> Draw a bootstrap sample: m rows chosen **with replacement** from the m training rows, so some appear more than once and some not at all. Fit a tree to it. Repeat `n_estimators` times. To predict, ask every tree and take the majority vote.

A note of history: bagging is Leo Breiman's, published in 1996 as "Bagging predictors" — the same Breiman who co-wrote the CART trees of 1984 and, five years later, the random forest.

Nothing about the tree itself changes — bagging reuses section 2's algorithm unmodified and changes only what surrounds it.

One detail of scikit-learn's `BaggingClassifier`: it does not count votes, it averages the trees' predicted probabilities. With fully grown trees the two are the same thing. Every leaf is pure, so each tree's probability is 0 or 1, and averaging 0s and 1s is counting votes. Notebook 2's own majority-vote version and scikit-learn's therefore apply the same rule, though not to the same trees: each draws its own bootstrap samples. They still make the same prediction for all 360 applicants of its test split, 0.872 accuracy both — with 100 trees, the luck of any one bootstrap sample is outvoted.

5.1 How much data one bootstrap sample actually sees

Each bootstrap sample is drawn from the same m rows with replacement, so the chance any one specific row is never drawn across m draws is

$$\left(1-\frac{1}{m}\right)^m$$

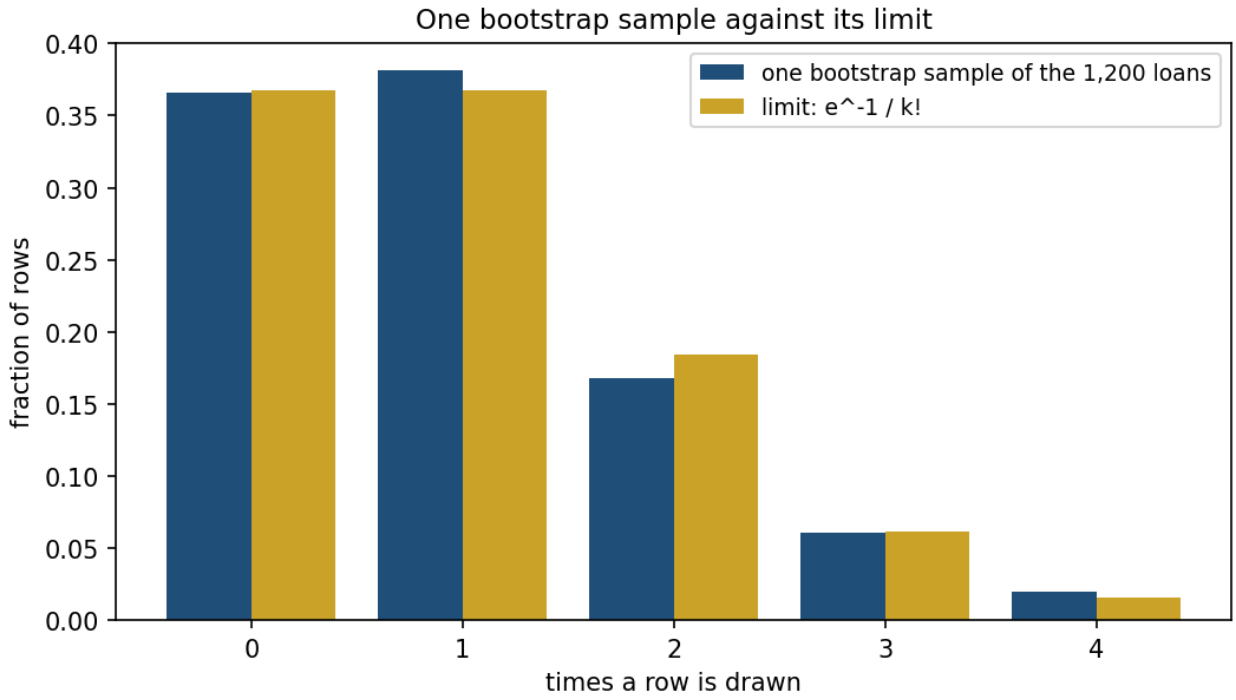
As $m \to \infty$, this is the standard exponential limit $\lim_{m\to\infty}\left(1+\frac{x}{m}\right)^m = e^x$ with $x=-1$:

$$\left(1-\frac{1}{m}\right)^m \xrightarrow{m\to\infty} e^{-1} \approx 0.368$$

so each bootstrap sample contains about $1-e^{-1} \approx 63.2\%$ of the distinct rows and leaves about **36.8%** out entirely — called **out-of-bag** (OOB). At $m = 1{,}200$ the finite-sample value is $(1-1/1200)^{1200} = 0.3677$, already within 0.0002 of the limit, and a single simulated bootstrap draw left out 36.6% of rows — close to both, with the residual gap being exactly the sampling noise a single draw carries.

A predictable mistake, and why it is reasonable. "So each tree is trained on 63.2% of the data, about 759 rows." The number is the one just derived, so the reading is natural. But a bootstrap sample still has $m = 1{,}200$ draws: about 759 *different* rows, and the other 441 draws are repeats of rows already drawn. The tree is fit to 1,200 rows, some of them two or three times over, which is what the next paragraph counts.

The same reasoning gives the whole distribution of how often a row is drawn: in the limit, a fraction $e^{-1}/k!$ of rows appears exactly k times — 36.8% never, 36.8% once, 18.4% twice, 6.1% three times. The number of times one row is drawn is binomial, m draws each with probability $1/m$; as m grows with the mean held at $m \cdot 1/m = 1$, the binomial tends to the Poisson distribution with mean 1, whose probabilities are exactly $e^{-1}/k!$.



One bootstrap sample of the 1,200 loans (blue) against the limit $e^{-1}/k!$ (gold). The first bar is the out-of-bag share; the rest are rows the tree sees once or several times. About one row in four is drawn two or more times, and that duplication is what makes each tree different from its neighbour.

Every OOB row is free validation data for the trees that did not see it. Predicting each row by a vote of only those trees, and scoring those predictions, gives the **OOB score**, with no rows set aside. On this dataset a 300-tree random forest — bagging with one extra rule, given in section 6; here it can be read as 300 bagged trees — had an OOB score of **0.9117**, against a 5-fold cross-validated accuracy of **0.9117 ± 0.021** for the same forest.

That the two match to four decimal places is this seed's luck, and it is worth saying so rather than letting the coincidence stand as the demonstration. Repeating both over twelve seeds gives a typical gap of **0.0026** and a worst case of **0.0075**; the exact four-decimal match turns up twice in the twelve. What the theory predicts is the weaker and more useful claim: the OOB score works like cross-validation — every row is scored only by models that never saw it — so the two estimate the same quantity and agree **to within their own noise**, about ±0.003 here against a cross-validation spread of ±0.021. The bookkeeping differs. In 5-fold cross-validation each fold is scored by a whole

forest trained without it. Out of bag, each row is scored by the trees whose bootstrap sample missed it: about a third of the forest, and a different third for each row. There is no single out-of-bag set.

The practical consequence is unchanged: with a bagged ensemble you get a trustworthy estimate of generalisation without setting aside a validation set at all. It is simply not an estimate to read to the fourth decimal.

5.2 Why averaging reduces variance, and what it cannot fix

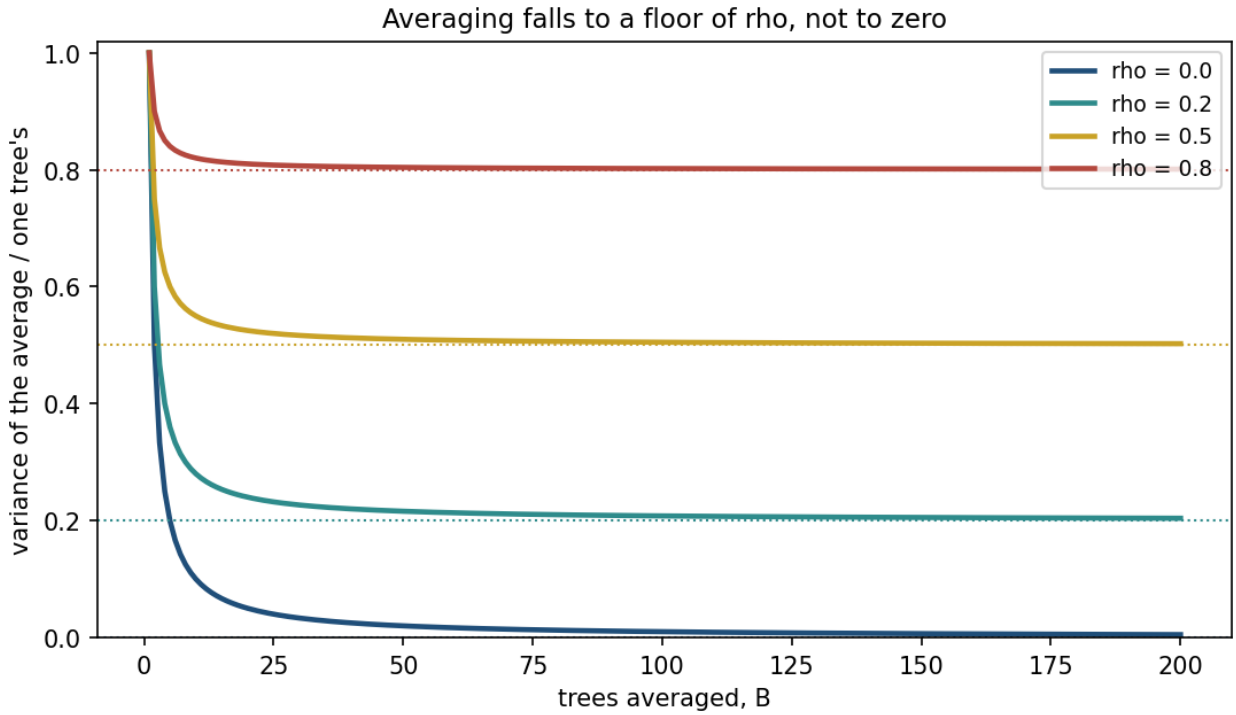
Model each tree's prediction as a random variable with variance σ^2 (over the randomness of which bootstrap sample it happened to see), and let ρ be the average correlation between the predictions of any two trees. For B trees averaged together:

$$\operatorname{Var}\left(\frac{1}{B}\sum_{b=1}^{B} f_b\right) = \frac{1}{B^2}\left(\sum_{b=1}^{B}\operatorname{Var}(f_b) + \sum_{b\neq b'}\operatorname{Cov}(f_b, f_{b'})\right)$$

There are B variance terms, each σ^2, and $B(B-1)$ covariance terms, each $\rho\sigma^2$ — the same accounting lesson 5, Section 4.3 did for k correlated fold scores, which is not a coincidence: averaging correlated measurements is one piece of arithmetic, and this is its second appearance in the course:

$$= \frac{1}{B^2}\Big(B\sigma^2 + B(B-1)\,\rho\sigma^2\Big) = \frac{\sigma^2}{B} + \frac{B-1}{B}\,\rho\sigma^2 \xrightarrow{B\to\infty} \rho\sigma^2$$

Two consequences follow directly from the algebra. First, variance falls towards a **floor** of $\rho\sigma^2$, not to zero: adding trees past the point where σ^2/B is small buys almost nothing, because the $\rho\sigma^2$ term does not shrink with B at all. Second, averaging **cannot repair an error every tree shares**. For an average of numbers this is exact: the mean of unbiased-but-noisy predictions is unbiased, and the mean of predictions that share a systematic error keeps that error. A vote between classifiers is not an average of numbers, and it can move a point across the boundary — which is why the worked example below sees the mean accuracy rise, not only the spread narrow. But no vote can fix a mistake every voter makes. Bagging is above all a variance tool. It does nothing for trees that are too shallow to represent the rule in the first place.



The variance of an average of B trees, as a share of one tree's, for four values of ρ. Uncorrelated trees average it away; correlated ones stop at the dotted floor however many are added — at $\rho = 0.5$, a hundred trees still keep 0.505 of one tree's variance. Lowering ρ is the only way through the floor, and it is what section 6 does.

Worked example. Across 30 independent 70/30 train/test splits of the loan data, a single unconstrained tree's test accuracy has mean **0.856** and standard deviation **0.0185**; a 100-tree bagged ensemble has mean **0.902** and standard deviation **0.0153**. Both numbers moved in the predicted direction — the mean rose because bagging also averages out some of a deep tree's overfitting, and the spread fell, as the formula above leads us to expect. The measurement is consistent with the formula rather than a direct check of it: the formula is about the variance of one prediction over bootstrap samples, while this is the spread of test accuracy over 30 different train/test splits, which also carries the luck of which applicants landed in each test set.

---

6. Random forests: decorrelating the trees

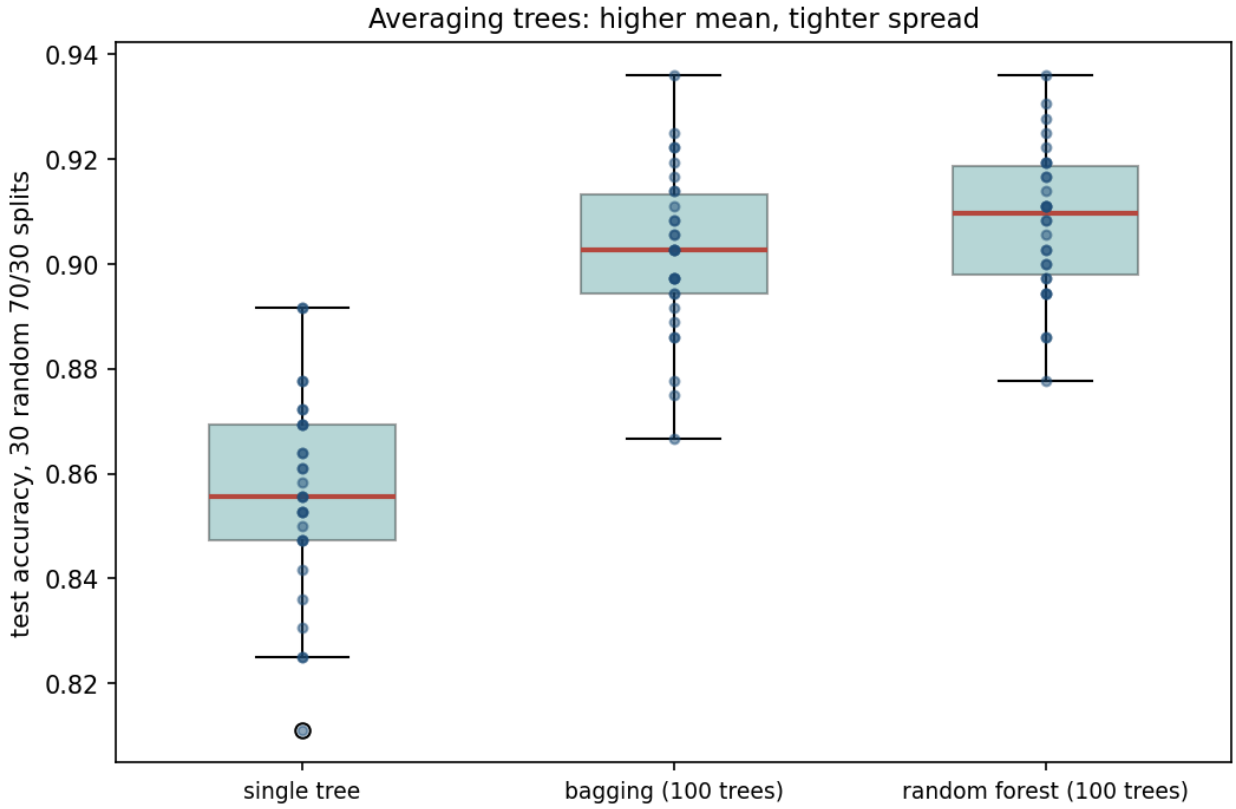
A note of history: Breiman's "Random Forests" appeared in 2001, the same year as his essay "Statistical Modeling: The Two Cultures", a defence of judging models by how well they predict.

The variance floor in section 5.2 is $\rho\sigma^2$, so shrinking it further means shrinking ρ — making the trees agree with each other less. Bootstrap resampling alone does this only partly, because bagged trees still consider every feature at every split, so a strong feature pulls most bootstrap trees towards splitting on it first regardless of which rows they drew.

A **random forest** adds one restriction: at each split, only a random subset of `max_features` features — by default $\lfloor\sqrt{n}\rfloor$ for classification — is even offered as a candidate. Different trees are forced to consider different features at the same point in the tree, which lowers ρ and lets the

variance floor drop further, without changing the splitting rule itself. It is bagging with the menu of candidate splits shrunk at random, every time a split is made.

Measured on the same 30 splits as section 5.2, a 100-tree random forest reaches mean accuracy **0.908** with standard deviation **0.0137** — both a better mean and a tighter spread than plain bagging's 0.902 and 0.0153, consistent with ρ having fallen further.



Test accuracy over 30 random 70/30 splits, single tree against bagging against random forest. Reading left to right: the median rises, the box narrows, and the whiskers pull in — mean and spread both improve together, exactly as sections 5.2 and 6 predicted from the $\rho\sigma^2$ floor.

---

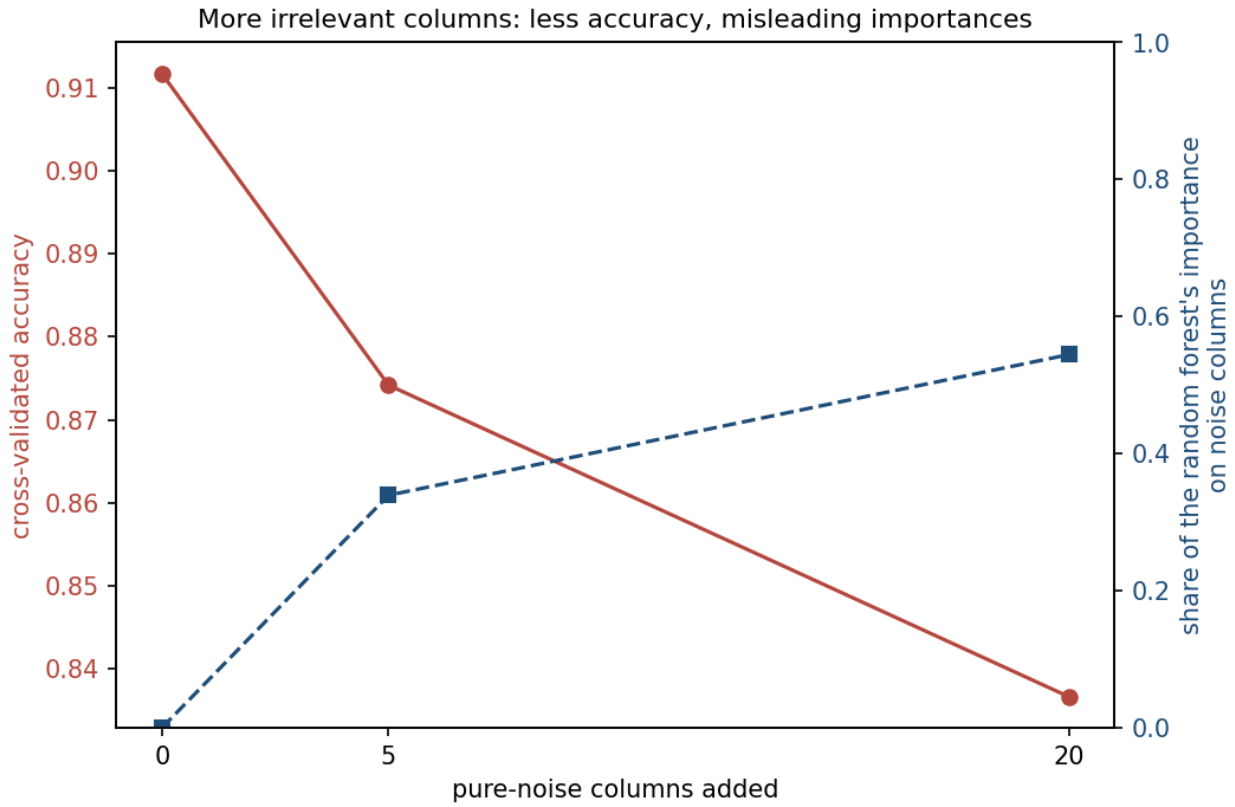
7. Feature importance, and its limits

A random forest reports **feature importance**: the total impurity reduction each feature is responsible for, summed and averaged across every tree and every split. It is tempting to read this as "the forest tells you which features matter" — and with only the two real features in this dataset, it does exactly that.

The question is what happens once most of the columns carry nothing.

Noise columns added	Total columns	Cross-validated accuracy	Importance on noise columns
0	2	0.912	0.0%
5	7	0.874	33.9%

Noise columns added	Total columns	Cross-validated accuracy	Importance on noise columns
20	22	0.837	**54.5%**



Cross-validated accuracy (left axis) falls as pure-noise columns are added; the share of the forest's own importance that lands on those noise columns (right axis) rises at the same time. The two lines crossing is the whole warning: accuracy degrading gracefully gives no hint that the importance ranking behind it has become largely wrong.

More irrelevant columns cost accuracy — the same direction as lesson 6's curse of dimensionality, though the mechanism is different — and, more strikingly, an increasing share of the forest's own importance scores lands on columns that are `np.random.normal` noise by construction, wired to nothing. **With 20 pure-noise columns against 2 real features, 54% of the forest's importance mass sits on noise.**

Two things hand credit to noise, and they are worth taking in order.

The menu. The first follows from section 6's restriction directly. With `max_features = "sqrt"` and $n = 22$ total columns, each split considers $k = \lfloor\sqrt{22}\rfloor = 4$ candidates. By symmetry — the number of size-k subsets of n features that contain one fixed feature is $\binom{n-1}{k-1}$, against $\binom{n}{k}$ subsets in total — any one specific column, including either real feature, is a candidate with probability

$$\frac{\binom{n-1}{k-1}}{\binom{n}{k}} = \frac{k}{n} = \frac{4}{22} \approx 18.2\%$$

The middle equality is worth one line of cancellation, since it is what makes the answer so simple: writing both binomials as factorials,

$$\frac{(n-1)!}{(k-1)!\,(n-k)!}\cdot\frac{k!\,(n-k)!}{n!}=\frac{k!}{(k-1)!}\cdot\frac{(n-1)!}{n!}=\frac{k}{n}$$

— every factorial cancels except one factor at each end.

So any one column is *excluded* from a given split with probability $1-4/22\approx 81.8\%$. For both real features to be left out together, all four candidates must come from the 20 noise columns, which happens with probability

$$\frac{\binom{20}{4}}{\binom{22}{4}}=\frac{4{,}845}{7{,}315}\approx 0.662$$

— about two splits in three offer no real feature at all, and the tree must choose its best option among whichever noise columns happened to be drawn.

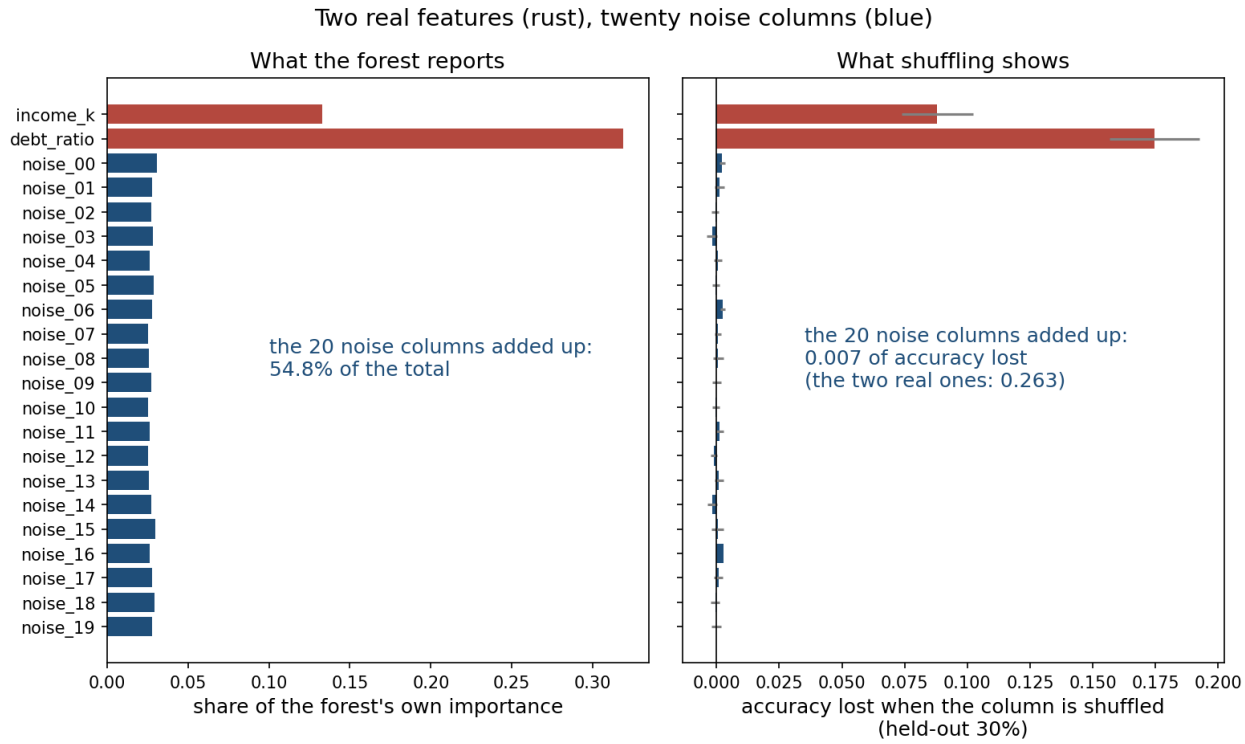
Small groups. The menu is not the whole story. Give every split all 22 columns, with `max_features=None`, and the noise share does not fall to zero: it falls from 54.5% to **28%**. The rest comes from the bottom of the trees. A node deep in a fully grown tree holds a handful of applicants, and among twenty columns of random numbers one will almost always separate that handful a little better than income or debt can, purely by chance. Each such gain is small, but it is not zero, and enough of them accumulate across hundreds of trees and thousands of splits to produce the shares measured above.

The same run says something about the default, too. With every column on offer, cross-validated accuracy rises from 0.837 to **0.897**: with 20 useless columns out of 22, the random menu costs more accuracy than decorrelating the trees buys back. `"sqrt"` is a good default, not a law, and the number of useless columns is what decides it.

The predictable mistake, and why the instinct is sound. It is entirely reasonable to expect an average over hundreds of trees to wash out noise — and section 5.2's variance-reduction result is real, measured, and points the same way. The averaging genuinely reduces the *variance of the prediction*. It does not make the credit for noise small, because both causes above act inside every tree, and averaging many trees that each credit noise gives an average that credits noise. A second, milder version of the same instinct: "with every feature on offer, noise gets nothing". It is sound near the root, where the real features win easily; it fails at the bottom of the trees, where the groups are small.

7.1 The check: permutation importance

The importance a forest reports is computed from its own training splits, so it is a claim the forest makes about itself. **Permutation importance** tests the claim from outside. Take the fitted model and data it was not trained on; shuffle the values of one column, so that the column no longer says anything about the applicant in its row; score the model again. The accuracy lost is that column's importance. A column the model relies on costs accuracy when shuffled; a column it never really used costs almost nothing.



The 20-noise-column forest, fit on 70% of the applicants and checked on the other 30%. Left: the importance the forest reports for itself, more than half of it on the twenty noise columns. Right: the same 22 columns by permutation on the held-out 30%. The two real features carry all of it; every noise column sits at zero, within a few thousandths either way.

Shuffling debt ratio costs 0.18 of accuracy, shuffling income 0.09, and each noise column between 0.003 lost and 0.002 gained. *Almost* nothing, not exactly nothing: the shuffle is random and the test set finite, so a useless column moves the score by a few thousandths in either direction. And the check has to run on held-out data. On the training rows a fully grown forest has memorised its noise columns along with everything else, and shuffling them costs it accuracy there — the same flaw as the built-in importance, measured a second way.

The practical consequence: with few real features against many candidate columns, treat a random forest's importances as suggestive, not definitive. Check them by permutation on held-out data — one call to `sklearn.inspection.permutation_importance` — and, before asking the forest which columns matter, ask which ones could plausibly matter at all.

> **Try this:** in notebook 2, section 6, run the permutation check on the *training* rows instead of the held-out ones, and measure how much the twenty noise columns cost there, added up, against the 0.007 they cost on held-out data. The paragraph above says which way it moves; the size is yours to find.

---

8. Gradient boosting: correcting mistakes in sequence

A note of history: the first boosting algorithm to work in practice was AdaBoost, by Yoav Freund and Robert Schapire (1995–1997), which reweighted the examples the trees so far got wrong; it won

them the Gödel Prize in 2003. Jerome Friedman's 2001 paper recast the idea as gradient descent on a loss — the version this section teaches and scikit-learn implements.

Bagging and random forests build trees **independently** and average at the end. **Boosting** builds trees **in sequence**: each new tree is fit specifically to correct what the trees so far got wrong.

$$F_0(x) = \bar{y}, \qquad F_t(x) = F_{t-1}(x) + \alpha \cdot h_t(x)$$

where h_t is a shallow tree — depth 2 or 3, a **weak learner** — fit to what remains unexplained by F_{t-1}, and α is the **learning rate**: how much of each new tree's correction is actually applied. F_0 is the best single constant for the loss being minimised: for squared error, the case of this section, it is the mean $\bar{y}$; for classification, section 9, it will be the log-odds of the base rate.

The trees are **regression trees**, even when the task is classification, because what they are fit to is a number, not a class. A regression tree is section 2's algorithm with two changes: each leaf predicts the mean of the values that reach it, and a split is scored by how much it lowers the squared error around those means, in place of the Gini impurity. The greedy search over features and thresholds is unchanged.

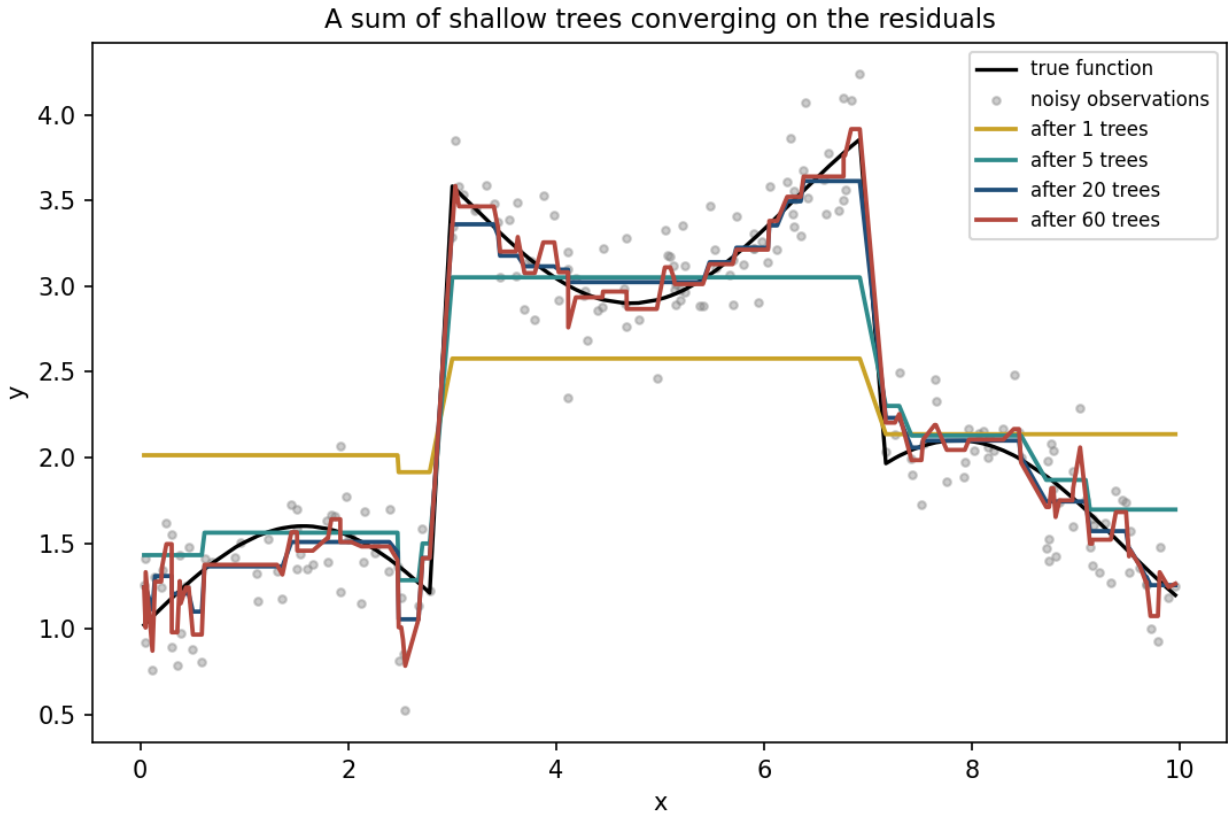
A predictable mistake, and why it is reasonable. "The final task is classification, so each boosted tree votes default or repaid, like a tree in a forest." Every ensemble before this one was made of trees that vote, so the expectation is natural. But a boosted tree never outputs a class. It outputs a correction that is added to a running score F, and only the final score is turned into a class.

For squared-error regression, "what remains unexplained" has an exact name, the **residual** $y - F_{t-1}(x)$. This is a special case of a more general idea — **functional gradient descent**: h_t is fit to approximate the negative gradient of the loss with respect to F_{t-1} at each training point. For $L(y, F) = \frac{1}{2}(y - F)^2$,

$$-\frac{\partial L}{\partial F} = -(F - y) = y - F$$

so "fit the residual" and "fit the negative gradient" coincide exactly for squared error, and the general recipe reduces to the familiar one.

Worked example. A synthetic target — a three-level step function plus a gentle sine wave, observed with noise — was fit by 60 depth-2 trees at $\alpha = 0.3$:



A sum of shallow trees converging on the residuals. One tree barely moves the flat starting guess (mean squared error, MSE, against the true function: 0.395). Five trees sketch the coarse shape (MSE 0.068). Twenty trace the step function closely, oscillation included (MSE 0.012) — the ensemble's best point. By sixty trees the ensemble is tracking individual noisy observations as well as the underlying shape, and the error against the (noise-free) truth has risen back to 0.018: the same overfitting story as section 3's unconstrained tree, reached by a different route — not one tree grown too deep, but too many shallow trees each chasing whatever residual noise is left.

---

9. Boosting for classification: the same idea, a different gradient

Classification does not have a "residual" in the same literal sense, but the functional-gradient recipe still applies — only the loss changes. With $y \in \{0, 1\}$ and the ensemble's output converted to a probability by the logistic function, $p = \sigma(F) = 1/(1 + e^{-F})$, the loss is the **log-loss** lesson 4 used for logistic regression:

$$L(y, F) = -[y \log p + (1 - y) \log(1 - p)]$$

By the chain rule, $\dfrac{\partial L}{\partial p} = -\dfrac{y}{p} + \dfrac{1 - y}{1 - p}$ and $\dfrac{\partial p}{\partial F} = p(1 - p)$ (the logistic function's own derivative), so

$$\frac{\partial L}{\partial F} = \frac{\partial L}{\partial p} \cdot \frac{\partial p}{\partial F} = \left(-\frac{y}{p} + \frac{1 - y}{1 - p} \right) p(1 - p) = p - y$$

and the negative gradient — the **pseudo-residual** each tree is fit to — is $y - p$: the true label minus the current predicted probability, exactly the error term lesson 4's logistic regression gradient descent moved against. Boosting for classification is fitting a sequence of small trees to that same quantity, one correction at a time, rather than adjusting a fixed set of linear coefficients against it in one continuous descent.

Where it starts, and the first residuals. The starting score is the best constant, the log-odds of the base rate: 464 of the 1,200 applicants defaulted, $p = 0.387$, so $F_0 = \log(464/736) = -0.461$, which the logistic function turns back into 0.387 for every applicant. The first tree is then fit to $y - p$: **+0.613** for every applicant who defaulted and **−0.387** for every one who repaid. Where that tree finds a group of mostly defaulters, it raises their score; each later tree works on what the earlier ones left.

A predictable mistake, and why it is reasonable. "So the update is $p + \alpha(y - p)$." The pseudo-residual is a difference of probabilities, so adding it to a probability looks natural. But the trees correct the score F, not p: F moves by $\alpha h_t(x)$, and only then does the logistic function turn the new F into a new p. That is what keeps p between 0 and 1 however large the corrections add up.

One detail of scikit-learn's implementation, for anyone who reads its source. The splits of each tree are chosen on $y - p$ exactly as described; the value written in each leaf is then recomputed as $\sum(y - p) \,/\, \sum p(1 - p)$ over the leaf's examples — one Newton step on the log-loss in place of the plain mean. The splits come from the residuals, the size of each step from the curvature of the loss. Nothing in this lesson depends on it.

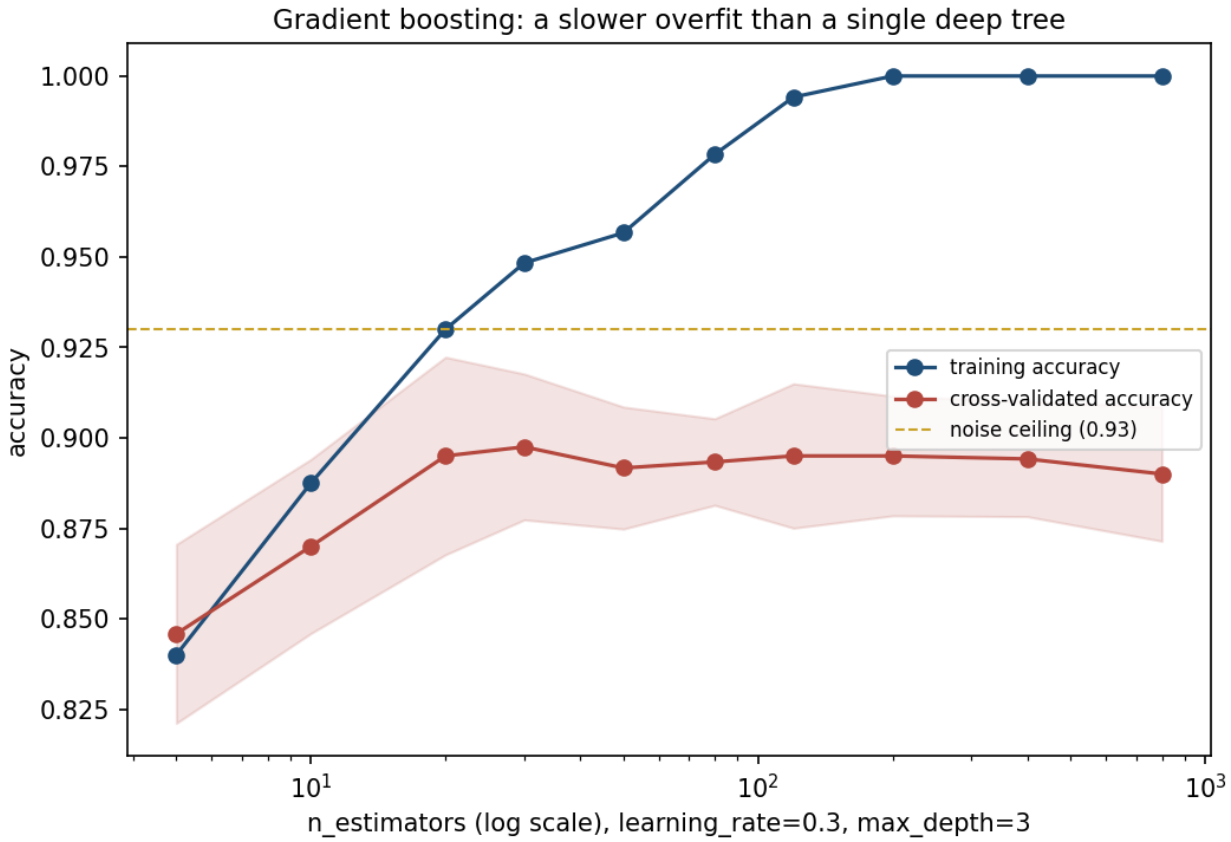
A default `GradientBoostingClassifier` — 100 trees, learning rate 0.1 — reaches **0.902 ± 0.020** cross-validated accuracy on the loan data, already ahead of any single tree.

---

10. Overfitting in boosting: two knobs that trade off

Unlike bagging, boosting keeps reducing its training loss for as long as it runs, because each tree is built specifically to reduce what remains. With nothing to stop it, that includes the 7% of labels that are noise by construction. The loss is the quantity that falls at every step; training accuracy follows it on the whole but not at every single tree — fit to all 1,200 applicants with 800 trees, it dips at a handful of steps, too few to show in the table.

`n_estimators` ($\alpha = 0.3$)	Training accuracy	Cross-validated accuracy
5	0.840	0.846 ± 0.025
10	0.888	0.870 ± 0.024
20	0.930	0.895 ± 0.027
30	0.948	**0.897 ± 0.020**
80	0.978	0.893 ± 0.012
200	1.000	0.895 ± 0.017
800	1.000	0.890 ± 0.019



Training accuracy reaches 1.000 by 200 trees and stays there — the same ceiling section 3's unconstrained tree reached in one step, here approached gradually. Cross-validated accuracy climbs quickly, peaks around 30 trees, and drifts down by roughly a point over the following 770 trees.

The overfitting is real but **gentler** than a single unconstrained tree's: that tree lost 3 points of cross-validated accuracy past its peak (section 3); 800 boosted trees lose about 1, even though training accuracy has been at 1.000 since well before that point. That gentleness is why the number of trees has to be chosen on held-out data rather than left to run — the damage per added tree is small and easy to miss on a single run, and it is still there. There are two ways to choose it. This lesson cross-validated a list of tree counts and kept the best. The other is **early stopping**: set aside part of the training data, watch the score on it as trees are added, and stop once it has not improved for a fixed number of rounds; in scikit-learn it is the `n_iter_no_change` argument of `GradientBoostingClassifier`.

Learning rate and tree count are not independent: both control how much total correction the ensemble applies, so they trade off against each other. Fixing `n_estimators = 120` and varying only α:

Learning rate α	Training accuracy	Cross-validated accuracy
0.02	0.888	0.872 ± 0.025
0.05	0.914	0.897 ± 0.022
0.10	0.939	**0.902 ± 0.022**
0.30	0.994	0.895 ± 0.020
1.00	1.000	0.878 ± 0.020

Learning rate α	Training accuracy	Cross-validated accuracy

A learning rate of 0.02 has not finished fitting the signal in 120 trees — training accuracy is still under 0.89. A learning rate of 1.0 has memorised the training set completely (1.000) and given back more than two points of cross-validated accuracy for it, 0.902 down to 0.878. The best setting sits in between: large enough to make real progress within the tree budget, small enough to leave each individual correction easy to outvote if it turns out to be wrong.

A predictable mistake, and why it is reasonable. "Only α times the number of trees matters, so half the learning rate with twice the trees gives the same model." The paragraph above has just said that both knobs set the total correction, so the shortcut is natural. But they are not interchangeable: each tree is fit to what the ensemble before it still gets wrong, so many small steps follow a different path from a few large ones. A large step overshoots on the noisy labels and the next trees spend themselves undoing it; a small one lets later trees outvote an early mistake. The trade-off is real; the equivalence is not.

---

11. Choosing among trees, forests and boosting

Every method from this lesson, on the same five folds:

Model	Cross-validated accuracy
Majority baseline	0.613
Logistic regression	0.748
Single tree, unconstrained	0.852
Single tree, `max_depth = 8`	0.882
Gradient boosting, 30 trees	0.897
Single tree, `max_leaf_nodes = 9`	0.901
Bagging, 100 trees	0.904
Random forest, 100 trees	**0.911**

Noise ceiling: 0.93 (1 − the 7% deliberately flipped labels). No method reaches it, and none should — a model that classified the flipped labels "correctly" would be modelling the noise, not the rule.

Three findings, read together, are this lesson's real content. First, on this dataset the ensembles — bagging, the random forest, gradient boosting — land within about a point and a half of each other: which ensembling *strategy* you pick matters less than whether you ensemble at all. Second, a single tree sits among them: the nine-leaf tree of section 4 reaches 0.901, above gradient boosting and a point behind bagging, and a person can read it start to finish. On data whose rule is a handful of axis-aligned rectangles, a small tree is nearly enough; what the ensembles buy here is less a higher score than a steadier one — section 5.2 measured how much less a forest's score moves from one split of the data to another, and section 4 showed where a single tree's verdicts wobble. Third, the unconstrained tree is the worst model on this list despite being, in a narrow sense, the most flexible one: flexibility without a stopping rule is not an advantage, a point this course has now made with several different methods.

Situation	Reach for
The decision must be explained to the person it affects	A single tree, small enough to read; if even that is too opaque, none of these models
Tabular data, accuracy matters most, no explanation required	A random forest: the best score here, at its default settings
Accuracy matters, and there is time to tune the number of trees and the learning rate together	Gradient boosting, tuned on validation data
Many columns, few of them informative	Any tree method, checking its importances by permutation (section 7.1): section 7's 54% came with 1,200 rows
Too few rows to set a validation set aside	A random forest, judged by its out-of-bag score

One caveat, in lesson 5's terms. Depth 8 and 30 boosted trees were chosen because they scored best on the same five folds as the leaderboard, so those two leaderboard rows are slightly optimistic; nested cross-validation would remove the bias. Bagging and the random forest ran at their defaults, chose nothing on these folds, and are not affected — and they are still the two best rows. The nine-leaf budget was chosen for readability, not on these folds.

Summary

- A decision tree splits greedily on Gini impurity, $G = 1 - \sum_c p_c^2$, one feature and threshold at a time — verified here against scikit-learn's implementation, prediction by prediction.
- Depth is the bias-variance dial. Cross-validated accuracy peaked at **depth 8** (0.882); an unconstrained tree reached training accuracy **1.000** and lost 3 points of cross-validated accuracy for it.
- A tree's strongest split survives resampling almost unchanged; its weaker splits do not: two trees grown on 65% samples gave opposite verdicts to 140 of the 1,200 applicants (11.7%), almost all around the islands. A budget of nine leaves (`max_leaf_nodes`) cross-validated at 0.901, above depth 8.
- Bagging averages trees fit to bootstrap resamples. Each sample leaves about **36.8%** of rows out-of-bag — the standard limit $(1 - 1/m)^m \to e^{-1}$ — which is why the OOB score is free validation; across twelve seeds it tracked 5-fold cross-validation to within 0.003.
- Averaged variance falls towards a floor of $\rho\sigma^2$, not zero. A random forest lowers ρ by restricting each split to a random subset of features, tightening test-accuracy spread from ± 0.019 (single tree) to ± 0.015 (bagging) to ± 0.014 (random forest) over 30 resampled splits. Averaging cannot repair an error every tree shares.
- **With 20 pure-noise columns against 2 real features, 54% of a random forest's feature importance landed on noise** — the number worth carrying out of this lesson. Two causes: `max_features="sqrt"` offers no real feature at two splits in three, and deep splits on small groups credit noise by chance even when every feature is on offer. Permutation importance on held-out data puts the noise columns back at zero.
- Gradient boosting fits shallow trees in sequence to the negative gradient of the loss — the residual for squared error, $y - p$ for log-loss, the same quantity lesson 4's logistic regression descended against. Cross-validated accuracy peaked near 30 trees and drifted down by about a point over the next 770, a gentler overfit than a single unconstrained tree's but not an absent

one.

- On this dataset the ensembles landed within about a point and a half of each other, the random forest on top at 0.911 — and the nine-leaf tree came within a point of bagging, because the rule is made of rectangles.

Homework

`Exercises/07_trees_and_ensembles.md`, discussed at the start of Lesson 8, **Friday 27 November 2026**.

Notation used in this lesson

Symbol	Meaning
G	Gini impurity of a group of examples
ΔG	impurity reduction a candidate split buys
p_c	fraction of a group belonging to class c
d	a tree's depth
m, n	number of examples, number of features
m_g; m_P, m_L, m_R	examples in a group; in a parent and its two children
σ^2, ρ	variance of one tree's prediction, correlation between two trees
k	features offered as candidates at each split (`max_features`)
B	number of trees averaged
F_t, h_t	the ensemble's prediction after t rounds, and the t-th tree added to it
α	**learning rate**, boosting's shrinkage factor — the same symbol as lessons 3 and 9
p	the model's predicted probability, $\sigma(F)$

Further reading

Resource	Type	Why read it
scikit-learn: decision trees	Official docs	The splitting criteria and stopping parameters stated precisely
scikit-learn: ensemble methods	Official docs	Bagging, random forests and boosting side by side, with the parameters that matter
Breiman, *Random Forests* (2001)	Paper	The original derivation of the $\rho\sigma^2$ variance bound used in section 5.2

Resource	Type	Why read it
Friedman, *Greedy Function Approximation: A Gradient Boosting Machine* (2001)	Paper	The functional-gradient view of boosting, section 8's general recipe, from its source
Hastie, Tibshirani & Friedman, *Elements of Statistical Learning*, ch. 9–10, 15	Book	Trees (ch. 9), boosting (ch. 10) and random forests (ch. 15), the definitive treatment
Molnar, *Interpretable Machine Learning*, ch. 5–6	Book, free online	Permutation importance and its failure modes — the fix section 7 points towards